

Splunk Data Source Preparation for UBA Integration

Table of Contents

Overview

Why Preparation Matters

Prerequisites

Splunk Environment Requirements

UBA Integration Requirements

CIM Compliance Requirements

Understanding CIM Compliance

Key CIM Data Models for UBA

Making Data CIM Compliant

Data Source Preparation by Type

Human Resources (HR) Data Preparation

Asset Data Preparation

Authentication Data Preparation

DNS Data Preparation (*High Priority - Identity Resolution*)

DHCP Data Preparation (*High Priority - Identity Resolution*)

Network Traffic Data Preparation

Email Data Preparation

Endpoint Data Preparation

VPN Data Preparation (*Medium Priority*)

Cloud Infrastructure Data Preparation

Search Query Preparation

Creating Efficient UBA Search Queries

Sample Search Queries by Data Source Type

Index Configuration

Index Planning for UBA Integration

Data Routing Configuration

Field Extraction and Normalization

Automatic Field Extraction

Data Normalization Examples

Data Quality Validation

Pre-Integration Data Quality Checks

Data Quality Reports

Performance Optimization

Search Performance Tuning

UBA-Specific Optimizations

Service Account Configuration

Splunk User Account Setup

SSL Certificate Configuration

Testing and Validation

Pre-Integration Testing

Integration Testing

References

Table of Sources

Splunk Data Source Preparation for UBA Integration

Table of Contents

1. Overview
2. Prerequisites
3. CIM Compliance Requirements
4. Data Source Preparation by Type
 - Human Resources (HR) Data Preparation (*Critical - First*)
 - Asset Data Preparation (*Critical - Second*)
 - Authentication Data Preparation (*High Priority*)
 - DNS Data Preparation (*High Priority*)
 - DHCP Data Preparation (*High Priority*)
 - Network Traffic Data Preparation (*Medium Priority*)
 - Endpoint Data Preparation (*Medium Priority*)
 - Email Data Preparation (*Medium Priority*)
 - VPN Data Preparation (*Medium Priority*)
 - Cloud Infrastructure Data Preparation (*As Needed*)
5. Search Query Preparation

6. Index Configuration
7. Field Extraction and Normalization
8. Data Quality Validation
9. Performance Optimization
10. Service Account Configuration
11. Testing and Validation
12. Common Preparation Tasks

Overview

Before adding data sources to UBA, proper preparation in Splunk Enterprise or Splunk Cloud is essential. This manual covers all necessary steps to ensure your Splunk data is properly formatted, indexed, and accessible for UBA integration.

Why Preparation Matters

- **Data Quality:** Ensures UBA receives clean, properly formatted data
- **Performance:** Optimizes data retrieval and processing
- **Compliance:** Meets CIM (Common Information Model) requirements
- **Reliability:** Reduces integration failures and data quality issues
- **Efficiency:** Minimizes troubleshooting and rework

Prerequisites

Splunk Environment Requirements

- ☐ Splunk Enterprise 8.0+ or Splunk Cloud Platform
- ☐ Administrative access to Splunk
- ☐ Knowledge of Splunk SPL (Search Processing Language)
- ☐ Access to data sources and indexes
- ☐ Understanding of your organization's data formats

UBA Integration Requirements

- ☐ Network connectivity from UBA to Splunk management port (8089)
- ☐ Splunk service account with appropriate permissions
- ☐ SSL certificates configured (if using HTTPS)
- ☐ Understanding of data volume and retention requirements

CIM Compliance Requirements

Understanding CIM Compliance

The Common Information Model (CIM) provides a standardized way to normalize data across different sources. UBA works best with CIM-compliant data.

CIM Benefits for UBA

- **Standardized Fields:** Consistent field names across data sources
- **Event Classification:** Proper event categorization
- **Tag Normalization:** Standardized tags for event types
- **Data Model Alignment:** Integration with Splunk ES and other tools

Key CIM Data Models for UBA

1. **Authentication:** Login/logout events, authentication failures
2. **Network Traffic:** Firewall, proxy, network communications
3. **Endpoint:** Host-based security events, process execution
4. **Email:** Email communications and metadata
5. **Web:** HTTP/HTTPS web traffic and proxy logs
6. **Change:** System and configuration changes
7. **Malware:** Antivirus and malware detection events

Making Data CIM Compliant

Method 1: Using Splunk Add-ons

```
# Install appropriate Splunk Add-ons for your data sources
# Common add-ons for UBA:
- Splunk Add-on for Microsoft Windows
- Splunk Add-on for Microsoft Active Directory
- Splunk Add-on for Palo Alto Networks
- Splunk Add-on for Cisco ASA
- Splunk Add-on for Microsoft Exchange
```

Method 2: Manual CIM Mapping

```
# Example: Making Windows Security Events CIM compliant
index=wineventlog source="WinEventLog:Security"
| eval action=case(
    EventCode=4624, "success",
    EventCode=4625, "failure",
    1=1, "unknown"
)
| eval src_user=coalesce(Account_Name, User_Name)
| eval dest=coalesce(Computer_Name, WorkstationName)
| eval app=coalesce(Process_Name, Logon_Process)
| fields _time, action, src_user, dest, app, EventCode
```

Method 3: Using Transforms and Field Extractions

```
# In transforms.conf
[extract_auth_fields]
REGEX = EventCode=(?<event_code>\d+).*Account_Name=(?<user>[^\s]+)
FORMAT = event_code::$1 user::$2

# In props.conf
[WinEventLog:Security]
TRANSFORMS-auth = extract_auth_fields
```

Data Source Preparation by Type

Human Resources (HR) Data Preparation

Employee Directory and Profile Data

```
# Prepare HR employee data for UBA
index=hr sourcetype=hr:employee_data
| eval employee_id=coalesce(emp_id, employee_id, employee_number)
| eval full_name=coalesce(full_name, first_name." ".last_name, display_name)
| eval username=lower(coalesce(username, user_id, login_id, email))
| eval email=lower(coalesce(email, email_address, work_email))
| eval department=coalesce(department, dept, division, business_unit)
| eval job_title=coalesce(job_title, title, position, role)
| eval manager=coalesce(manager, supervisor, manager_name, reports_to)
| eval hire_date=coalesce(hire_date, start_date, employment_start)
| eval employment_status=case(
    match(employment_status, "(?i)active"), "active",
    match(employment_status, "(?i)terminated"), "terminated",
    match(employment_status, "(?i)suspended"), "suspended",
    match(employment_status, "(?i)leave"), "on_leave",
    1=1, "unknown"
)
| eval location=coalesce(location, office_location, work_location, site)
| eval employee_type=coalesce(employee_type, employment_type, worker_type)
| eval security_clearance=coalesce(security_clearance, clearance_level)
| eval cost_center=coalesce(cost_center, cost_centre, budget_code)
| fields _time, employee_id, username, full_name, email, department, job_title, manager, hire_date, employment_status, location, employee_type, security_clearance, cost_center
```

HR Organizational Changes

```
# Prepare HR organizational change events for UBA
index=hr sourcetype=hr:org_changes
| eval employee_id=coalesce(emp_id, employee_id, employee_number)
| eval username=lower(coalesce(username, user_id, login_id))
| eval change_type=case(
    match(change_type, "(?i)hire"), "new_hire",
    match(change_type, "(?i)promotion"), "promotion",
    match(change_type, "(?i)transfer"), "transfer",
    match(change_type, "(?i)termination"), "termination",
    match(change_type, "(?i)department.*change"), "department_change",
    match(change_type, "(?i)title.*change"), "title_change",
    match(change_type, "(?i)manager.*change"), "manager_change",
    1=1, "other"
)
| eval old_value=coalesce(old_value, previous_value, from_value)
| eval new_value=coalesce(new_value, current_value, to_value)
| eval effective_date=coalesce(effective_date, change_date, implementation_date)
| eval changed_by=coalesce(changed_by, modified_by, hr_rep)
| eval approval_status=coalesce(approval_status, status, approval_state)
| fields _time, employee_id, username, change_type, old_value, new_value, effective_date, changed_by, approval_status
```

Access Provisioning/Deprovisioning Events

```
# Prepare HR access management events for UBA
index=hr sourcetype=hr:access_management
| eval employee_id=coalesce(emp_id, employee_id, employee_number)
| eval username=lower(coalesce(username, user_id, login_id))
| eval action=case(
    match(action, "(?i)provision"), "provision",
    match(action, "(?i)deprovision"), "deprovision",
    match(action, "(?i)modify"), "modify",
    match(action, "(?i)suspend"), "suspend",
    match(action, "(?i)reactivate"), "reactivate",
    1=1, "unknown"
)
| eval system_name=coalesce(system_name, target_system, application)
| eval access_level=coalesce(access_level, permission_level, role_assigned)
| eval requested_by=coalesce(requested_by, requester, manager)
| eval approved_by=coalesce(approved_by, approver, hr_rep)
| eval request_date=coalesce(request_date, created_date)
| eval completion_date=coalesce(completion_date, processed_date)
| eval reason=coalesce(reason, justification, business_reason)
| fields _time, employee_id, username, action, system_name, access_level, requested_by, approved_by, request_date, completion_date, reason
```

Official Splunk HR Data Example

The following SPL example is based on the official Splunk UBA documentation for extracting HR data using LDAP:

```
# Official Splunk UBA HR data extraction using ldapsearch
# Based on Splunk UBA Admin Guide - HR Data Source Configuration
| inputlookup ldap_users.csv
| eval username=lower(sAMAccountName)
| eval email=lower(mail)
| eval full_name=displayName
| eval job_title=title
| eval department=department
| eval manager=manager
| eval employee_id=employeeID
| eval employment_status=case(
    userAccountControl="512", "active",
    userAccountControl="514", "disabled",
    1=1, "unknown"
)
| eval location=physicalDeliveryOfficeName
| eval hire_date=strftime(strptime(whenCreated, "%Y%m%d%H%M%S.%fZ"), "%Y-%m-%d")
| eval last_login=strftime(strptime(lastLogon, "%Y%m%d%H%M%S.%fZ"), "%Y-%m-%d %H:%M:%S")
| eval cost_center=extensionAttribute1
| eval employee_type=extensionAttribute2
| eval security_clearance=extensionAttribute3
| fields username, email, full_name, job_title, department, manager, employee_id, employment_status, location, hire_date, last_login, cost_center,
employee_type, security_clearance
| outputlookup hr_data_for_uba.csv
```

Note: This example requires: - LDAP integration configured in Splunk - Proper field mapping based on your Active Directory schema - Regular scheduled searches to keep HR data current - Reference: Splunk UBA Admin Guide, Chapter 5: Data Sources

HR Data Validation Query

```
# Validate HR data quality for UBA integration
| inputlookup hr_data_for_uba.csv
| eval issues=""
| eval issues=if(isnull(username) OR username="", issues."Missing username; ", issues)
| eval issues=if(isnull(email) OR email="", issues."Missing email; ", issues)
| eval issues=if(isnull(full_name) OR full_name="", issues."Missing full name; ", issues)
| eval issues=if(isnull(department) OR department="", issues."Missing department; ", issues)
| eval issues=if(isnull(employment_status) OR employment_status="", issues."Missing employment_status; ", issues)
| eval data_quality=case(
    issues="", "Good",
    len(issues)<50, "Needs Attention",
    1=1, "Poor"
)
| stats count by data_quality, issues
| sort -count
```

Asset Data Preparation

Asset Inventory and Classification

```
# Prepare asset inventory data for UBA
index=assets sourcetype=asset:inventory
| eval asset_id=coalesce(asset_id, asset_tag, serial_number, device_id)
| eval hostname=lower(coalesce(hostname, computer_name, device_name, asset_name))
| eval ip_address=coalesce(ip_address, primary_ip, network_address)
| eval mac_address=upper(coalesce(mac_address, physical_address, nic_address))
| eval asset_type=case(
    match(asset_type, "(?i)laptop"), "laptop",
    match(asset_type, "(?i)desktop"), "desktop",
    match(asset_type, "(?i)server"), "server",
    match(asset_type, "(?i)mobile"), "mobile_device",
    match(asset_type, "(?i)tablet"), "tablet",
    match(asset_type, "(?i)printer"), "printer",
    match(asset_type, "(?i)network"), "network_device",
    1=1, "unknown"
)
| eval operating_system=coalesce(os, operating_system, platform)
| eval os_version=coalesce(os_version, version, build)
| eval owner=coalesce(owner, assigned_to, user, primary_user)
| eval department=coalesce(department, dept, division, cost_center)
| eval location=coalesce(location, site, building, office)
| eval criticality=case(
    match(criticality, "(?i)critical"), "critical",
    match(criticality, "(?i)high"), "high",
    match(criticality, "(?i)medium"), "medium",
    match(criticality, "(?i)low"), "low",
    1=1, "unknown"
)
| eval asset_status=case(
    match(status, "(?i)active"), "active",
    match(status, "(?i)inactive"), "inactive",
    match(status, "(?i)retired"), "retired",
    match(status, "(?i)maintenance"), "maintenance",
    1=1, "unknown"
)
| eval last_seen=coalesce(last_seen, last_update, last_scan)
| eval data_type=case(
    sourcetype="asset:inventory", "asset_inventory",
    sourcetype="asset:config_changes", "configuration_change",
    sourcetype="asset:software", "software_inventory",
    1=1, "asset_other"
)
| fields _time, asset_id, hostname, ip_address, mac_address, asset_type, operating_system, os_version, owner, department, location, criticality, asset_status, last_seen, data_type
```

Asset Configuration Changes

```
# Prepare asset configuration change events for UBA
index=assets sourcetype=asset:config_changes
| eval asset_id=coalesce(asset_id, asset_tag, device_id)
| eval hostname=lower(coalesce(hostname, computer_name, device_name))
| eval change_type=case(
    match(change_type, "(?i)software.*install"), "software_install",
    match(change_type, "(?i)software.*remove"), "software_removal",
    match(change_type, "(?i)config.*change"), "configuration_change",
    match(change_type, "(?i)hardware.*change"), "hardware_change",
    match(change_type, "(?i)os.*update"), "os_update",
    match(change_type, "(?i)patch"), "patch_install",
    1=1, "other"
)
| eval changed_component=coalesce(component, changed_item, configuration_item)
| eval old_value=coalesce(old_value, previous_value, before)
| eval new_value=coalesce(new_value, current_value, after)
| eval changed_by=coalesce(changed_by, user, administrator, modified_by)
| eval change_source=coalesce(change_source, source_system, method)
| eval approval_required=coalesce(approval_required, requires_approval)
| eval approved_by=coalesce(approved_by, approver)
| eval risk_level=case(
    match(risk_level, "(?i)high"), "high",
    match(risk_level, "(?i)medium"), "medium",
    match(risk_level, "(?i)low"), "low",
    1=1, "unknown"
)
| fields _time, asset_id, hostname, change_type, changed_component, old_value, new_value, changed_by, change_source, approval_required, approved_by, risk_level
```

Software Inventory and Licensing

```
# Prepare software inventory data for UBA
index=assets sourcetype=asset:software
| eval asset_id=coalesce(asset_id, asset_tag, device_id)
| eval hostname=lower(coalesce(hostname, computer_name, device_name))
| eval software_name=coalesce(software_name, application_name, product_name)
| eval software_version=coalesce(version, software_version, product_version)
| eval vendor=coalesce(vendor, manufacturer, publisher)
| eval install_date=coalesce(install_date, installation_date, first_seen)
| eval last_used=coalesce(last_used, last_accessed, last_run)
| eval license_type=case(
    match(license_type, "(?i)commercial"), "commercial",
    match(license_type, "(?i)open.*source"), "open_source",
    match(license_type, "(?i)trial"), "trial",
    match(license_type, "(?i)shareware"), "shareware",
    match(license_type, "(?i)freeware"), "freeware",
    1=1, "unknown"
)
| eval installation_source=coalesce(installation_source, source, install_method)
| eval software_category=coalesce(category, software_type, classification)
| eval risk_rating=case(
    match(risk_rating, "(?i)high"), "high",
    match(risk_rating, "(?i)medium"), "medium",
    match(risk_rating, "(?i)low"), "low",
    1=1, "unknown"
)
| eval compliance_status=coalesce(compliance_status, license_compliance)
| fields _time, asset_id, hostname, software_name, software_version, vendor, install_date, last_used, license_type, installation_source, software_category, risk_rating, compliance_status
```

Authentication Data Preparation

Windows Active Directory Events

```
# Prepare Windows Security Events for UBA
index=vineventlog source="WinEventLog:Security"
(EventCode=4624 OR EventCode=4625 OR EventCode=4768 OR EventCode=4769 OR EventCode=4776 OR EventCode=4672 OR EventCode=4673)
| eval dataFormat="AD"
| eval user=coalesce(Account Name, Target Account Name, Service Account)
| eval src_ip=coalesce(Source_Network_Address, Client_Address, Workstation_IP_Address)
| eval dest=coalesce(Computer_Name, Target_Server_Name, Machine_Name)
| eval signature=EventCode
| eval action=case(
    EventCode=4624, "success",
    EventCode=4625, "failure",
    EventCode=4768, "success",
    EventCode=4769, "success",
    EventCode=4776, "success",
    1=1, "unknown"
)
| fields _time, user, src_ip, dest, signature, action, EventCode, dataFormat
```

VPN Authentication Events

```
# Prepare VPN logs for UBA
index=vpn sourcetype=cisco:asa
| eval action=case(
    match(_raw, "Built"), "success",
    match(_raw, "Deny"), "failure",
    match(_raw, "Teardown"), "logout",
    1=1, "unknown"
)
| eval user=if(isnotnull(user), user, "unknown")
| eval src_ip=coalesce(src_ip, orig_src_ip)
| eval dest_ip=coalesce(dest_ip, orig_dest_ip)
| eval login_server_type="VPN"
| fields _time, user, src_ip, dest_ip, action, login_server_type
```

DNS Data Preparation (*High Priority - Identity Resolution*)

Why High Priority: DNS logs provide critical user-to-IP mapping for identity resolution, enabling UBA to correlate user activities across different systems.

DNS Query Logs

```
# Prepare DNS query logs for UBA
index=network sourcetype=dns OR sourcetype=bind OR sourcetype=infoblox:dns
| eval query_name=lower(coalesce(query, question, name, domain))
| eval query_type=coalesce(query_type, qtype, record_type)
| eval response_code=coalesce(response_code, rcode, result_code)
| eval client_ip=coalesce(src_ip, client_ip, source_ip, orig_h)
| eval dns_server=coalesce(dest_ip, server_ip, dns_server, resp_h)
| eval response_time=coalesce(response_time, duration, query_time)
| eval answer=coalesce(answer, response, resolved_ip)
| eval action=case(
    response_code="0", "success",
    response_code="NOERROR", "success",
    response_code="3", "nxdomain",
    response_code="NXDOMAIN", "nxdomain",
    response_code="2", "servfail",
    response_code="SERVFAIL", "servfail",
    1=1, "unknown"
)
| eval query_category=case(
    match(query_name, ".*\.(com|net|org|edu|gov)$"), "external",
    match(query_name, ".*\.local$"), "internal",
    match(query_name, ".*\.(corp|internal|lan|domain)$"), "internal",
    1=1, "unknown"
)
| eval suspicious_indicators=case(
    len(query_name)>50, "long_domain_name",
    match(query_name, "^[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+$"), "reverse_lookup",
    match(query_name, ".*[0-9]{8}.*"), "suspicious_pattern",
    1=1, "normal"
)
| fields _time, client_ip, dns_server, query_name, query_type, response_code, action, answer, response_time, query_category, suspicious_indicators
```

DNS Response Analysis

```
# Analyze DNS responses for anomaly detection
index=network sourcetype=dns
| eval client_ip=coalesce(src_ip, client_ip, source_ip)
| eval query_name=lower(coalesce(query, question, name))
| eval answer=coalesce(answer, response, resolved_ip)
| stats count as query_count,
    dc(query_name) as unique_domains,
    dc(answer) as unique_responses,
    values(query_name) as domains_queried
    by client_ip
| eval dns_activity_score=case(
    query_count>1000 AND unique_domains>500, "high",
    query_count>500 AND unique_domains>250, "medium",
    query_count>100 AND unique_domains>50, "low",
    1=1, "normal"
)
| where dns_activity_score!="normal"
| sort -query_count
```

DHCP Data Preparation *(High Priority - Identity Resolution)*

Why High Priority: DHCP logs provide IP-to-hostname mapping essential for correlating network activities with specific devices and users.

DHCP Lease Events

```
# Prepare DHCP lease events for UBA
index=network sourcetype=dhcp OR sourcetype=cisco:dhcp OR sourcetype=windows:dhcp
| eval action=case(
    match(_raw, "(?i)ACK"), "lease_granted",
    match(_raw, "(?i)DISCOVER"), "discover",
    match(_raw, "(?i)REQUEST"), "request",
    match(_raw, "(?i)RELEASE"), "release",
    match(_raw, "(?i)DECLINE"), "decline",
    match(_raw, "(?i)NAK"), "nak",
    1=1, "unknown"
)
| eval client_ip=coalesce(assigned_ip, client_ip, yiaddr, ip_address)
| eval client_mac=upper(coalesce(client_mac, mac_address, chaddr, mac))
| eval client_hostname=lower(coalesce(client_hostname, hostname, client_name))
| eval dhcp_server=coalesce(server_ip, dhcp_server, siaddr, server)
| eval lease_duration=coalesce(lease_time, lease_duration, lease_seconds)
| eval vendor_class=coalesce(vendor_class, option_60, vendor_identifier)
| eval relay_agent=coalesce(relay_agent, giaddr, relay_ip)
| eval subnet_mask=coalesce(subnet_mask, option_1, netmask)
| eval default_gateway=coalesce(default_gateway, option_3, gateway, router)
| eval dns_servers=coalesce(dns_servers, option_6, name_servers)
| eval lease_type=case(
    action="lease_granted", "active",
    action="release", "released",
    action="nak", "denied",
    1=1, "pending"
)
| fields _time, client_ip, client_mac, client_hostname, dhcp_server, action, lease_duration, vendor_class, relay_agent, subnet_mask, default_gateway, dns_servers, lease_type
```

DHCP Lease Correlation

```
# Create IP-to-device mapping from DHCP leases
index=network sourcetype=dhcp action="lease_granted"
| eval client_ip=coalesce(assigned_ip, client_ip, yiaddr)
| eval client_mac=upper(coalesce(client_mac, mac_address, chaddr))
| eval client_hostname=lower(coalesce(client_hostname, hostname, client_name))
| sort 0 _time
| streamstats current=f last(client_hostname) as prev_hostname,
    last(client_mac) as prev_mac by client_ip
| eval hostname_change=if(client_hostname!=prev_hostname AND isnotnull(prev_hostname), "yes", "no")
| eval mac_change=if(client_mac!=prev_mac AND isnotnull(prev_mac), "yes", "no")
| eval lease_anomaly=case(
    hostname_change="yes" AND mac_change="yes", "device_replacement",
    hostname_change="yes" AND mac_change="no", "hostname_change",
    hostname_change="no" AND mac_change="yes", "mac_change",
    1=1, "normal"
)
| where lease_anomaly!="normal"
| fields _time, client_ip, client_mac, client_hostname, prev_hostname, prev_mac, lease_anomaly
```

Network Traffic Data Preparation

Firewall Data

```
# Prepare firewall logs for UBA
index=firewall sourcetype=pan:traffic
| eval action=case(
    action="allow", "allowed",
    action="deny", "blocked",
    action="drop", "blocked",
    1=1, action
)
| eval src_zone=coalesce(src_zone, from_zone)
| eval dest_zone=coalesce(dest_zone, to_zone)
| eval protocol=upper(protocol)
| eval bytes_in=coalesce(bytes_in, bytes_received)
| eval bytes_out=coalesce(bytes_out, bytes_sent)
| fields _time, src_ip, dest_ip, src_port, dest_port, protocol, action, src_zone, dest_zone, bytes_in, bytes_out, app
```

DNS Data

```
# Prepare DNS logs for UBA
index=dns sourcetype=dns
| eval query_type=upper(query_type)
| eval response_code=coalesce(response_code, rcode)
| eval query=lower(query)
| eval answer=coalesce(answer, resolved_ip)
| eval src_ip=coalesce(src_ip, client_ip, source_ip)
| fields _time, src_ip, query, query_type, response_code, answer, dest_ip
```

Email Data Preparation

Microsoft Exchange Logs

```
# Prepare Exchange message tracking logs for UBA
index=email sourcetype=msexchange:messagetracking
| eval sender=lower(sender)
| eval recipients=lower(recipients)
| eval subject=if(len(subject)>100, substr(subject,1,100)."...", subject)
| eval message_size=coalesce(total_bytes, size)
| eval direction=case(
    match(source_context, "SMTP"), "inbound",
    match(source_context, "MAPI"), "outbound",
    1=1, "internal"
)
| fields _time, sender, recipients, subject, message_size, direction, message_id
```

Endpoint Data Preparation

Windows Process Events

```
# Prepare Windows process creation events for UBA
index=wineventlog source="WinEventLog:Security" EventCode=4688
| eval process_name=coalesce(New_Process_Name, Process_Name)
| eval parent_process=coalesce(Creator_Process_Name, Parent_Process_Name)
| eval user=coalesce(Subject_Account_Name, Account_Name)
| eval dest=coalesce(Computer_Name, WorkstationName)
| eval command_line=coalesce(Process_Command_Line, Command_Line)
| fields _time, user, dest, process_name, parent_process, command_line, EventCode
```

USB/Removable Media Events


```
# Prepare USB/removable media events for UBA
index=endpoint sourcetype=symantec:ep:risk:file
| eval device_type=case(
    match(file_path, "(?i)removable"), "USB",
    match(file_path, "(?i)usb"), "USB",
    match(file_path, "(?i)floppy"), "Floppy",
    1=1, "Unknown"
)
| eval user=coalesce(user_name, username, user)
| eval file_name=coalesce(file_name, filename)
| eval action=coalesce(action, event_type)
| fields _time, user, device_type, file_name, file_path, action, computer_name
```

VPN Data Preparation (*Medium Priority*)

Why Medium Priority: VPN logs provide remote access patterns and help identify unusual connection behaviors and potential compromise indicators.

VPN Connection Events

```
# Prepare VPN connection logs for UBA
index=network sourcetype=cisco:asa OR sourcetype=paloalto:vpn OR sourcetype=fortinet:vpn OR sourcetype=checkpoint:vpn
| eval user=lower(coalesce(user, username, vpn_user, account_name))
| eval src_ip=coalesce(src_ip, client_ip, external_ip, remote_ip)
| eval vpn_server=coalesce(dest_ip, server_ip, vpn_gateway, local_ip)
| eval action=case(
    match(_raw, "(?i)connect"), "connect",
    match(_raw, "(?i)disconnect"), "disconnect",
    match(_raw, "(?i)login"), "login",
    match(_raw, "(?i)logout"), "logout",
    match(_raw, "(?i)auth.*success"), "auth_success",
    match(_raw, "(?i)auth.*fail"), "auth_failure",
    1=1, "unknown"
)
| eval vpn_type=case(
    match(sourcetype, "asa"), "ssl_vpn",
    match(sourcetype, "checkpoint"), "ipsec_vpn",
    match(_raw, "(?i)ssl"), "ssl_vpn",
    match(_raw, "(?i)ipsec"), "ipsec_vpn",
    match(_raw, "(?i)l2tp"), "l2tp_vpn",
    1=1, "unknown"
)
| eval session_id=coalesce(session_id, connection_id, tunnel_id)
| eval session_duration=coalesce(duration, session_time, connect_time)
| eval bytes_in=coalesce(bytes_in, rx_bytes, inbound_bytes)
| eval bytes_out=coalesce(bytes_out, tx_bytes, outbound_bytes)
| eval assigned_ip=coalesce(assigned_ip, internal_ip, tunnel_ip)
| eval authentication_method=coalesce(auth_method, authentication_type, auth_protocol)
| eval client_version=coalesce(client_version, user_agent, vpn_client)
| eval failure_reason=coalesce(failure_reason, error_reason, reject_reason)
| fields _time, user, src_ip, vpn_server, assigned_ip, action, vpn_type, session_id, session_duration, bytes_in, bytes_out, authentication_method, client_version, failure_reason
```

VPN Session Analysis

```
# Analyze VPN session patterns for anomaly detection
index=network sourcetype=*vpn* action="connect"
| eval user=lower(coalesce(user, username, vpn_user))
| eval src_ip=coalesce(src_ip, client_ip, external_ip)
| eval session_start=_time
| eval hour_of_day=strftime(_time, "%H")
| eval day_of_week=strftime(_time, "%w")
| eval is_weekend=if(day_of_week="0" OR day_of_week="6", "yes", "no")
| eval is_off_hours=if(hour_of_day<"06" OR hour_of_day>"22", "yes", "no")
| stats count as connection_count,
    dc(src_ip) as unique_source_ips,
    values(src_ip) as source_ips,
    dc(hour_of_day) as time_variance,
    sum(eval(if(is_weekend="yes",1,0))) as weekend_connections,
    sum(eval(if(is_off_hours="yes",1,0))) as off_hours_connections
    by user
| eval anomaly_score=case(
    unique_source_ips>5 AND connection_count>20, "high",
    unique_source_ips>3 AND weekend_connections>5, "medium",
    off_hours_connections>10, "medium",
    time_variance>12, "low",
    1=1, "normal"
)
| where anomaly_score!="normal"
| sort -connection_count
```

VPN Geographic Analysis

```
# Analyze VPN connections by geographic location
index=network sourcetype=*vpn* action="connect"
| eval user=lower(coalesce(user, username, vpn_user))
| eval src_ip=coalesce(src_ip, client_ip, external_ip)
| iplocation src_ip
| eval country=if(isnull(Country), "Unknown", Country)
| eval city=if(isnull(City), "Unknown", City)
| stats count as connections,
    earliest(_time) as first_seen,
    latest(_time) as last_seen,
    dc(src_ip) as unique_ips
    by user, country, city
| eval location_risk=case(
    match(country, "(?i)(china|russia|iran|north korea)"), "high",
    match(country, "(?i)unknown"), "medium",
    connections<5, "low",
    1=1, "normal"
)
| eval time_span=round((last_seen-first_seen)/86400, 2)
| where location_risk!="normal" OR connections>50
| sort -connections
```

Cloud Infrastructure Data Preparation

AWS CloudTrail Events

```
# Prepare AWS CloudTrail events for UBA
index=aws sourcetype=aws:cloudtrail
| eval aws_account_id=coalesce(recipientAccountId, account)
| eval user_name=coalesce(userName, user)
| eval user_type=coalesce(userIdentity.type, identity_type)
| eval source_ip=coalesce(sourceIPAddress, src_ip)
| eval user_agent=coalesce(userAgent, user_agent)
| eval event_name=coalesce(eventName, event)
| eval event_source=coalesce(eventSource, service)
| eval aws_region=coalesce(awsRegion, region)
| eval error_code=coalesce(errorCode, error)
| eval error_message=coalesce(errorMessage, error_msg)
| eval event_time=coalesce(eventTime, _time)
| eval session_context=coalesce('userIdentity.sessionContext.sessionIssuer.userName', session_user)
| eval mfa_used=case(
    'userIdentity.sessionContext.attributes.mfaAuthenticated'="true", "yes",
    'userIdentity.sessionContext.attributes.mfaAuthenticated'="false", "no",
    1=1, "unknown"
)
| fields _time, aws_account_id, user_name, user_type, source_ip, user_agent, event_name, event_source, aws_region, error_code, error_message, session_context, mfa_used
```

Azure Active Directory Events

```
# Prepare Azure AD events for UBA
index=azure sourcetype=azure:aad:audit
| eval user_principal_name=lower(coalesce('userPrincipalName', 'initiatedBy.user.userPrincipalName', user))
| eval display_name=coalesce('displayName', 'initiatedBy.user.displayName', display_name)
| eval activity_name=coalesce('activityDisplayName', activity, operation)
| eval category=coalesce('category', event_category)
| eval result_status=case(
    match('result', "(?i)success"), "success",
    match('result', "(?i)failure"), "failure",
    match('result', "(?i)interrupted"), "interrupted",
    1=1, "unknown"
)
| eval source_ip=coalesce('initiatedBy.user.ipAddress', src_ip, client_ip)
| eval user_agent=coalesce('additionalDetails{}.value', user_agent)
| eval target_resources=coalesce('targetResources{}.displayName', target)
| eval correlation_id=coalesce('correlationId', correlation_id)
| eval risk_level=coalesce('riskLevelAggregated', risk_level)
| eval app_name=coalesce('initiatedBy.app.displayName', application)
| fields _time, user_principal_name, display_name, activity_name, category, result_status, source_ip, user_agent, target_resources, correlation_id, risk_level, app_name
```

Search Query Preparation

Creating Efficient UBA Search Queries

1. Time Range Optimization

```
# Use specific time ranges to reduce search scope
earliest=-24h@h latest=@h
# Or use time-based filtering
| where _time >= relative_time(now(), "-24h@h")
```

2. Index and Sourcetype Filtering

```
# Always specify index and sourcetype for performance
index=wineventlog sourcetype="WinEventLog:Security"
# Multiple indexes
(index=wineventlog OR index=firewall OR index=dns)
```

3. Event Filtering

```
# Filter events early in the search pipeline
index=wineventlog sourcetype="WinEventLog:Security"
(EventCode=4624 OR EventCode=4625 OR EventCode=4768 OR EventCode=4769 OR EventCode=4776)
# Use NOT to exclude unwanted events
NOT (EventCode=4634 OR EventCode=4647)
```

4. Field Standardization

```
# Standardize field names for UBA consumption
| eval uba_user=coalesce(user, User_Name, Account_Name, src_user)
| eval uba_src_ip=coalesce(src_ip, Source_Network_Address, client_ip)
| eval uba_dest=coalesce(dest, Computer_Name, dest_host)
| eval uba_action=case(
  action="success" OR EventCode=4624, "success",
  action="failure" OR EventCode=4625, "failure",
  1=1, "unknown"
)
```

Sample Search Queries by Data Source Type

Authentication Search Query

```
index=wineventlog source="WinEventLog:Security"
(EventCode=4624 OR EventCode=4625 OR EventCode=4768 OR EventCode=4769 OR EventCode=4776 OR EventCode=4672 OR EventCode=4673)
| eval dataFormat="AD"
| eval user=coalesce(Account_Name, Target_Account_Name, Service_Account)
| eval src_ip=coalesce(Source_Network_Address, Client_Address, Workstation_IP_Address)
| eval dest=coalesce(Computer_Name, Target_Server_Name, Machine_Name)
| eval signature=EventCode
| eval login_type=coalesce(Logon_Type, Authentication_Type)
| eval action=case(
  EventCode=4624, "success",
  EventCode=4625, "failure",
  EventCode=4768, "success",
  EventCode=4769, "success",
  EventCode=4776, "success",
  EventCode=4672, "success",
  EventCode=4673, "success",
  1=1, "unknown"
)
| eval event_category=case(
  EventCode=4624 OR EventCode=4625, "authentication",
  EventCode=4768 OR EventCode=4769, "kerberos",
  EventCode=4776, "credential_validation",
  EventCode=4672 OR EventCode=4673, "privilege_use",
  1=1, "other"
)
| fields _time, user, src_ip, dest, signature, action, login_type, event_category, dataFormat, EventCode
```

Network Traffic Search Query

```
index=firewall sourcetype=pan:traffic
| eval src_ip=coalesce(src_ip, source_ip)
| eval dest_ip=coalesce(dest_ip, destination_ip, dest)
| eval src_port=coalesce(src_port, source_port)
| eval dest_port=coalesce(dest_port, destination_port)
| eval action=case(
  action="allow", "allowed",
  action="deny", "blocked",
  action="drop", "blocked",
  1=1, action
)
| eval protocol=upper(protocol)
| eval bytes_in=coalesce(bytes_in, bytes_received, received_bytes)
| eval bytes_out=coalesce(bytes_out, bytes_sent, sent_bytes)
| eval src_zone=coalesce(src_zone, from_zone, source_zone)
| eval dest_zone=coalesce(dest_zone, to_zone, destination_zone)
| eval session_id=coalesce(session_id, sessionid)
| eval transport=coalesce(transport, ip_protocol)
| fields _time, src_ip, dest_ip, src_port, dest_port, protocol, action, bytes_in, bytes_out, src_zone, dest_zone, app, session_id, transport
```

DNS Search Query

```
index=dns sourcetype=bind:query
| eval src_ip=coalesce(src_ip, client_ip, source_ip)
| eval query=lower(coalesce(query, domain, question))
| eval query_type=upper(coalesce(query_type, record_type, qtype))
| eval response_code=coalesce(response_code, rcode, return_code)
| eval answer=coalesce(answer, resolved_ip, response)
| eval dns_server=coalesce(dest_ip, server_ip, dns_server)
| eval query_time=coalesce(query_time, response_time)
| fields _time, src_ip, query, query_type, response_code, answer, dns_server, query_time
```

Email Search Query

```

index=email sourcetype=msexchange:messagetracking
| eval sender=lower(coalesce(sender, from, sender_address))
| eval recipients=lower(coalesce(recipients, to, recipient_address))
| eval subject=coalesce(subject, message_subject)
| eval message_id=coalesce(message_id, messageid, internal_message_id)
| eval message_size=coalesce(total_bytes, size, message_size)
| eval direction=case(
    match(source_context, "(?i)smtp"), "inbound",
    match(source_context, "(?i)mapi"), "outbound",
    match(source_context, "(?i)store"), "internal",
    1=1, "unknown"
)
| eval event_type=coalesce(event_id, eventid, event_type)
| eval server_hostname=coalesce(server_hostname, hostname, computer_name)
| fields _time, sender, recipients, subject, message_id, message_size, direction, event_type, server_hostname

```

VPN Search Query

```

index=vpn sourcetype=cisco:asa
| eval user=coalesce(user, username, vpn_user)
| eval src_ip=coalesce(src_ip, orig_src_ip, external_ip)
| eval dest_ip=coalesce(dest_ip, orig_dest_ip, internal_ip)
| eval action=case(
    match(_raw, "(?i)built"), "success",
    match(_raw, "(?i)deny"), "failure",
    match(_raw, "(?i)teardown"), "logout",
    match(_raw, "(?i)login"), "success",
    1=1, "unknown"
)
| eval login_server_type="VPN"
| eval session_id=coalesce(connection_id, session_id)
| eval vpn_protocol=coalesce(protocol, vpn_protocol)
| eval bytes_in=coalesce(bytes_in, received_bytes)
| eval bytes_out=coalesce(bytes_out, sent_bytes)
| fields _time, user, src_ip, dest_ip, action, login_server_type, session_id, vpn_protocol, bytes_in, bytes_out

```

HR Data Search Query

```

# Comprehensive HR search for UBA consumption
index=hr (sourcetype=hr:employee_data OR sourcetype=hr:org_changes OR sourcetype=hr:access_management)
| eval employee_id=coalesce(emp_id, employee_id, employee_number)
| eval username=lower(coalesce(username, user_id, login_id, email))
| eval full_name=coalesce(full_name, first_name, ".last_name, display_name)
| eval email=lower(coalesce(email, email_address, work_email))
| eval department=coalesce(department, dept, division, business_unit)
| eval job_title=coalesce(job_title, title, position, role)
| eval manager=coalesce(manager, supervisor, manager_name, reports_to)
| eval employment_status=case(
    match(employment_status, "(?i)active"), "active",
    match(employment_status, "(?i)terminated"), "terminated",
    match(employment_status, "(?i)suspended"), "suspended",
    match(employment_status, "(?i)leave"), "on_leave",
    1=1, "unknown"
)
| eval location=coalesce(location, office_location, work_location, site)
| eval hire_date=coalesce(hire_date, start_date, employment_start)
| eval termination_date=coalesce(termination_date, end_date, employment_end)
| eval security_clearance=coalesce(security_clearance, clearance_level)
| eval cost_center=coalesce(cost_center, cost_centre, budget_code)
| eval data_type=case(
    sourcetype="hr:employee_data", "employee_profile",
    sourcetype="hr:org_changes", "organizational_change",
    sourcetype="hr:access_management", "access_provisioning",
    1=1, "hr_other"
)
| fields _time, employee_id, username, full_name, email, department, job_title, manager, employment_status, location, hire_date, termination_date, security_clearance, cost_center, data_type

```

Asset Data Search Query

```
# Comprehensive Asset search for UBA consumption
index=assets (sourcetype=asset:inventory OR sourcetype=asset:config_changes OR sourcetype=asset:software)
| eval asset_id=coalesce(asset_id, asset_tag, serial_number, device_id)
| eval hostname=lower(coalesce(hostname, computer_name, device_name, asset_name))
| eval ip_address=coalesce(ip_address, primary_ip, network_address)
| eval mac_address=upper(coalesce(mac_address, physical_address, nic_address))
| eval asset_type=case(
    match(asset_type, "(?i)laptop"), "laptop",
    match(asset_type, "(?i)desktop"), "desktop",
    match(asset_type, "(?i)server"), "server",
    match(asset_type, "(?i)mobile"), "mobile_device",
    match(asset_type, "(?i)tablet"), "tablet",
    match(asset_type, "(?i)printer"), "printer",
    match(asset_type, "(?i)network"), "network_device",
    1=1, "unknown"
)
| eval operating_system=coalesce(os, operating_system, platform)
| eval os_version=coalesce(os_version, version, build)
| eval owner=coalesce(owner, assigned_to, user, primary_user)
| eval department=coalesce(department, dept, division, cost_center)
| eval location=coalesce(location, site, building, office)
| eval criticality=case(
    match(criticality, "(?i)critical"), "critical",
    match(criticality, "(?i)high"), "high",
    match(criticality, "(?i)medium"), "medium",
    match(criticality, "(?i)low"), "low",
    1=1, "unknown"
)
| eval asset_status=case(
    match(status, "(?i)active"), "active",
    match(status, "(?i)inactive"), "inactive",
    match(status, "(?i)retired"), "retired",
    match(status, "(?i)maintenance"), "maintenance",
    1=1, "unknown"
)
| eval last_seen=coalesce(last_seen, last_update, last_scan)
| eval data_type=case(
    sourcetype="asset:inventory", "asset_inventory",
    sourcetype="asset:config_changes", "configuration_change",
    sourcetype="asset:software", "software_inventory",
    1=1, "asset_other"
)
| fields _time, asset_id, hostname, ip_address, mac_address, asset_type, operating_system, os_version, owner, department, location, criticality, asset_status, last_seen, data_type
```

Cloud Infrastructure Search Query

```
# Comprehensive Cloud Infrastructure search for UBA consumption
(index=aws sourcetype=aws:cloudtrail) OR (index=azure sourcetype=azure:aad:audit) OR (index=gcp sourcetype=gcp:audit)
| eval platform=case(
    index="aws", "AWS",
    index="azure", "Azure",
    index="gcp", "GCP",
    1=1, "Unknown"
)
| eval user_name=case(
    platform="AWS", coalesce(userName, user),
    platform="Azure", lower(coalesce('userPrincipalName', 'initiatedBy.user.userPrincipalName', user)),
    platform="GCP", coalesce('protoPayload.authenticationInfo.principalEmail', user),
    1=1, "unknown"
)
| eval source_ip=case(
    platform="AWS", coalesce(sourceIPAddress, src_ip),
    platform="Azure", coalesce('initiatedBy.user.ipAddress', src_ip, client_ip),
    platform="GCP", coalesce('protoPayload.requestMetadata.callerIp', src_ip),
    1=1, "unknown"
)
| eval event_name=case(
    platform="AWS", coalesce(eventName, event),
    platform="Azure", coalesce('activityDisplayName', activity, operation),
    platform="GCP", coalesce('protoPayload.methodName', method),
    1=1, "unknown"
)
| eval event_result=case(
    platform="AWS" AND isnotnull(errorCode), "failure",
    platform="AWS" AND isnull(errorCode), "success",
    platform="Azure" AND match('result', "(?i)success"), "success",
    platform="Azure" AND match('result', "(?i)failure"), "failure",
    platform="GCP" AND 'protoPayload.response.error.code'>0, "failure",
    platform="GCP" AND 'protoPayload.response.error.code'=0, "success",
    1=1, "unknown"
)
| eval account_id=case(
    platform="AWS", coalesce(recipientAccountId, account),
    platform="Azure", coalesce('tenantId', tenant),
    platform="GCP", coalesce('resource.labels.project_id', project),
    1=1, "unknown"
)
| eval region=case(
    platform="AWS", coalesce(awsRegion, region),
    platform="Azure", coalesce('location', region),
    platform="GCP", coalesce('resource.labels.zone', 'resource.labels.region', region),
    1=1, "unknown"
)
| fields _time, platform, user_name, source_ip, event_name, event_result, account_id, region
```

Index Configuration

Index Planning for UBA Integration

1. Dedicated Indexes for UBA Data

```
# indexes.conf
[uba_authentication]
homePath = $SPLUNK_DB/uba_authentication/db
coldPath = $SPLUNK_DB/uba_authentication/colddb
thawedPath = $SPLUNK_DB/uba_authentication/thaweddb
maxDataSize = 10000
maxHotBuckets = 10
maxWarmDBCount = 300

[uba_network]
homePath = $SPLUNK_DB/uba_network/db
coldPath = $SPLUNK_DB/uba_network/colddb
thawedPath = $SPLUNK_DB/uba_network/thaweddb
maxDataSize = 10000
maxHotBuckets = 10
maxWarmDBCount = 300

[uba_endpoint]
homePath = $SPLUNK_DB/uba_endpoint/db
coldPath = $SPLUNK_DB/uba_endpoint/colddb
thawedPath = $SPLUNK_DB/uba_endpoint/thaweddb
maxDataSize = 10000
maxHotBuckets = 10
maxWarmDBCount = 300
```

2. Index Retention Planning

```
# Configure retention based on UBA requirements
# UBA typically needs 90-365 days of data
maxGlobalDataSizeMB = 500000
frozenTimePeriodInSecs = 31536000 # 365 days
```

3. Index Performance Optimization

```
# Optimize for UBA search patterns
maxHotSpanSecs = 7776000 # 90 days in hot buckets
maxMemMB = 20
maxConcurrentOptimizes = 6
```

Data Routing Configuration

1. props.conf for Data Source Types

```
# props.conf
[WinEventLog:Security]
SHOULD_LINEMERGE = false
TIME_PREFIX = SystemTime='
TIME_FORMAT = %Y-%m-%d %H:%M:%S
MAX_TIMESTAMP_LOOKAHEAD = 32
TRUNCATE = 65536
KV_MODE = xml

[cisco:asa]
SHOULD_LINEMERGE = false
TIME_PREFIX = ^
TIME_FORMAT = %b %d %H:%M:%S
MAX_TIMESTAMP_LOOKAHEAD = 32
TRUNCATE = 65536

[dns]
SHOULD_LINEMERGE = false
TIME_PREFIX = ^
TIME_FORMAT = %d-%b-%Y %H:%M:%S.%3N
MAX_TIMESTAMP_LOOKAHEAD = 32
TRUNCATE = 65536
```

2. transforms.conf for Field Extractions

```
# transforms.conf
[extract_windows_auth_fields]
REGEX = EventCode=(?<event_code>\d+).*Account_Name=(?<account_name>[^\s+]).*Source_Network_Address=(?<source_ip>[^\s+])
FORMAT = event_code::$1 account_name::$2 source_ip::$3

[extract_firewall_fields]
REGEX = src=(?<src_ip>\d+\.\d+\.\d+\.\d+)\s+dst=(?<dest_ip>\d+\.\d+\.\d+\.\d+)\s+sport=(?<src_port>\d+)\s+dport=(?<dest_port>\d+)
FORMAT = src_ip::$1 dest_ip::$2 src_port::$3 dest_port::$4
```

Field Extraction and Normalization

Automatic Field Extraction

1. Using Search-Time Extractions

```
# Create field extractions in Splunk Web
# Settings > Fields > Field extractions > New Field Extraction

# Example for Windows Event Log
EXTRACT-windows_user = Account_Name=(?<windows_user>[^\s]+)
EXTRACT-windows_computer = Computer_Name=(?<windows_computer>[^\s]+)
EXTRACT-event_code = EventCode=(?<event_code>\d+)
```

2. Using Calculated Fields

```
# Settings > Fields > Calculated fields > New Calculated field

# Normalize user names
if(match(user, "^[^\\]+\\\\(.+)$"), replace(user, "^[^\\]+\\\\(.+)$", "\1"), user)

# Standardize IP addresses
if(match(src_ip, "^\d+\.\d+\.\d+\.\d+$"), src_ip, null())

# Categorize event types
case(EventCode>=4624 AND EventCode<=4625, "authentication", EventCode=4688, "process", 1=1, "other")
```

Data Normalization Examples

User Name Normalization

```
# Handle different user name formats
| eval normalized_user=case(
  match(user, "^[^@]+@[^@]+$"), lower(user), # Email format
  match(user, "^[^\\]+\\\\(.+)$"), lower(replace(user, "^[^\\]+\\\\(.+)$", "\1")), # Domain\user
  match(user, "^(.+)[^@]+$"), lower(replace(user, "^(.+)[^@]+$", "\1")), # user@domain
  1=1, lower(user)
)
```

IP Address Normalization

```
# Standardize IP address formats
| eval normalized_src_ip=case(
  match(src_ip, "^\d+\.\d+\.\d+\.\d+$"), src_ip, # Valid IPv4
  match(src_ip, "^[0-9a-fA-F:]+$"), src_ip, # IPv6
  src_ip="-" OR src_ip="", null(), # Empty or dash
  1=1, null() # Invalid format
)
```

Hostname Normalization

```
# Normalize hostnames and computer names
| eval normalized_host=case(
  match(dest, "^[^.]+\.(.+)$"), lower(dest), # FQDN
  match(dest, "^[A-Za-z0-9\-.]+$"), lower(dest), # Simple hostname
  1=1, lower(dest)
)
| eval short_hostname=if(match(normalized_host, "^(.[^.]+)\."), replace(normalized_host, "^(.[^.]+)\.+$", "\1"), normalized_host)
```

Data Quality Validation

Pre-Integration Data Quality Checks

1. Data Completeness Validation

```
# Check for required fields in authentication data
index=wineventlog source="WinEventLog:Security" EventCode=4624
| eval has_user=if(isnotnull(Account_Name) AND Account_Name!="", 1, 0)
| eval has_src_ip=if(isnotnull(Source_Network_Address) AND Source_Network_Address!="" AND Source_Network_Address!="-", 1, 0)
| eval has_computer=if(isnotnull(Computer_Name) AND Computer_Name!="", 1, 0)
| stats count,
  avg(has_user) as pct_with_user,
  avg(has_src_ip) as pct_with_src_ip,
  avg(has_computer) as pct_with_computer
| eval data_quality_score=(pct_with_user + pct_with_src_ip + pct_with_computer)/3*100
```

2. Data Consistency Validation

```
# Check for consistent field formats
index=wineventlog source="WinEventLog:Security" EventCode=4624
| eval user_format=case(
  match(Account_Name, "^[^@]+@[^@]+$"), "email",
  match(Account_Name, "^[^\\]+\\\\(.+)$"), "domain\\user",
  match(Account_Name, "^[A-Za-z0-9\-.]+$"), "username",
  1=1, "other"
)
| stats count by user_format
| eval percentage=round(count/sum(count)*100, 2)
```

3. Data Volume Validation

```
# Check data volume trends
index=wineventlog source="WinEventLog:Security" earliest=-7d
| bucket _time span=1d
| stats count by _time, EventCode
| eval day=strftime(_time, "%Y-%m-%d")
| stats avg(count) as avg_events, stdev(count) as stdev_events by EventCode
| eval coefficient_of_variation=round(stdev_events/avg_events*100, 2)
```

4. Field Value Distribution

```
# Analyze field value distributions
index=wineventlog source="WinEventLog:Security" EventCode=4624
| stats count by Logon_Type
| sort -count
| eval percentage=round(count/sum(count)*100, 2)
```

Data Quality Reports

Daily Data Quality Report

```
# Create a daily data quality dashboard
| multisearch
  [search index=wineventlog sourcetype="WinEventLog:Security" earliest=-24h | eval source="Windows Security"]
  [search index=firewall earliest=-24h | eval source="Firewall"]
  [search index=dns earliest=-24h | eval source="DNS"]
  [search index=email earliest=-24h | eval source="Email"]
| stats count by source
| eval expected_min_events=case(
  source="Windows Security", 10000,
  source="Firewall", 50000,
  source="DNS", 100000,
  source="Email", 1000,
  1=1, 0
)
| eval status=if(count >= expected_min_events, "OK", "LOW")
| table source, count, expected_min_events, status
```

Weekly Data Quality Report

```
# Create a weekly data quality dashboard
| multisearch
  [search index=wineventlog sourcetype="WinEventLog:Security" earliest=-7d | eval source="Windows Security"]
  [search index=firewall earliest=-7d | eval source="Firewall"]
  [search index=dns earliest=-7d | eval source="DNS"]
  [search index=email earliest=-7d | eval source="Email"]
| stats count by source
| eval expected_min_events=case(
  source="Windows Security", 70000,
  source="Firewall", 350000,
  source="DNS", 700000,
  source="Email", 7000,
  1=1, 0
)
| eval status=if(count >= expected_min_events, "OK", "LOW")
| table source, count, expected_min_events, status
```

Performance Optimization

Search Performance Tuning

1. Index Time vs Search Time Operations

```
# Prefer index-time field extractions for frequently used fields
# In props.conf:
EXTRACT-user = Account_Name=(?<user>[^\s]+)
EXTRACT-src_ip = Source_Network_Address=(?<src_ip>[^\s]+)

# Use search-time operations for complex transformations
| eval normalized_user=lower(replace(user, "^[^\\]\\]+\\\\\\(.+)$", "\\1"))
```

2. Efficient Search Patterns

```
# Good: Specify index and sourcetype early
index=wineventlog sourcetype="WinEventLog:Security" EventCode=4624
| eval user=Account_Name
| stats count by user

# Better: Use metadata where possible
| metadata type=sourcetypes index=wineventlog
| where totalCount > 0

# Best: Use tstats for statistical operations
| tstats count where index=wineventlog sourcetype="WinEventLog:Security" by _time span=1h
```

3. Memory Optimization


```
# Limit result sets early in the pipeline
index=wineventlog sourcetype="WinEventLog:Security" EventCode=4624
| head 100000 # Limit results early
| eval user=Account_Name
| stats count by user
| sort -count
| head 100 # Final result limit
```

UBA-Specific Optimizations

1. Time Window Optimization

```
# UBA typically processes data in batches
# Optimize for 15-minute to 1-hour windows
earliest=-1h@h latest=@h

# For backfill operations, use larger windows
earliest=-24h@h latest=@h
```

2. Field Selection Optimization

```
# Only include fields that UBA needs
index=wineventlog source="WinEventLog:Security" EventCode=4624
| fields _time, Account_Name, Source_Network_Address, Computer_Name, EventCode, Logon_Type
```

3. Data Aggregation

```
# Pre-aggregate data when appropriate
index=firewall earliest=-1h@h latest=@h
| stats sum(bytes_in) as total_bytes_in,
        sum(bytes_out) as total_bytes_out,
        count as session_count
    by src_ip, dest_ip, src_port, dest_port, app
```

Service Account Configuration

Splunk User Account Setup

1. Create UBA Service Account

```
# In Splunk Web: Settings > Access controls > Users > New User
Username: uba_service
Full Name: UBA Integration Service Account
Email: uba-service@company.com
Default app: search
Time zone: inherit
```

2. Role Configuration

```
# Create custom role for UBA integration
# Settings > Access controls > Roles > New Role

Role name: uba_integration_role
Capabilities:
- search
- can_delete (for data cleanup if needed)
- list_default_indexes
- rest_properties_get
- rest_properties_set

Indexes:
- wineventlog (can search)
- firewall (can search)
- dns (can search)
- email (can search)
- vpn (can search)
- endpoint (can search)

Search restrictions:
- Maximum concurrent searches: 10
- Maximum search time: 1h
- Search quota: 100
```

3. Authentication Configuration

```
# Test service account authentication
curl -k -u uba_service:password \
  "https://splunk-server:8089/services/auth/login" \
  -d "username=uba_service&password=password"

# Verify search capabilities
curl -k -u uba_service:password \
  "https://splunk-server:8089/services/search/jobs" \
  -d "search=search index=wineventlog | head 10"
```

SSL Certificate Configuration

1. Export Splunk Server Certificate

```
# On Splunk server
cd /opt/splunk/etc/auth
cp server.pem uba-splunk-cert.pem
scp uba-splunk-cert.pem uba-server:/tmp/
```

2. Import Certificate to UBA

```
# On UBA server
. /opt/caspida/bin/CaspidaCommonEnv.sh
sudo keytool -import -alias splunk-server \
  -keystore $JAVA_HOME/lib/security/cacerts \
  -storepass changeit \
  -file /tmp/uba-splunk-cert.pem \
  -noprompt
```

3. Verify SSL Configuration

```
# Test SSL connection from UBA to Splunk
openssl s_client -connect splunk-server:8089 -verify_return_error
```

Testing and Validation

Pre-Integration Testing

1. Data Availability Testing

```
# Test data availability for the last 24 hours
| multisearch
  [search index=wineventlog sourcetype="WinEventLog:Security" earliest=-24h | eval source="Windows Security"]
  [search index=firewall earliest=-24h | eval source="Firewall"]
  [search index=dns earliest=-24h | eval source="DNS"]
  [search index=email earliest=-24h | eval source="Email"]
| stats count by source
| eval status=if(count > 0, "Available", "No Data")
| table source, count, status
```

2. Field Mapping Validation

```
# Validate field mappings for authentication data
index=wineventlog source="WinEventLog:Security" EventCode=4624 earliest=-1h
| eval uba_user=coalesce(Account_Name, Target_Account_Name, Service_Account)
| eval uba_src_ip=coalesce(Source_Network_Address, Client_Address)
| eval uba_dest=coalesce(Computer_Name, Target_Server_Name)
| eval uba_action="success"
| eval dataFormat="AD"
| stats count by uba_user, uba_src_ip, uba_dest, uba_action, dataFormat
| head 20
```

3. Search Performance Testing

```
# Test search performance for UBA-style queries
| rest /services/search/jobs
| search isSaved=0 AND isFinalized=1
| eval search_duration=runDuration
| eval search_type=case(
  match(search, "(?i)(EventCode|src_ip|dest_ip|user|account_name)"), "UBA",
  match(search, "(?i)(powershell|cmd|bash|sh)$"), "Shell",
  match(search, "(?i)(outlook|thunderbird|chrome|firefox|ie|explore)"), "User_App",
  match(search, "(?i)(net\\.exe|sc\\.exe|wmic|reg\\.exe)"), "Admin_Tool",
  match(search, "(?i)(psexec|wmiexec|winrs)"), "Remote_Admin",
  1=1, "Other"
)
| stats avg(search_duration) as avg_duration, max(search_duration) as max_duration, count
| eval performance_rating=case(
  avg_duration <= 30, "Excellent",
  avg_duration <= 60, "Good",
  avg_duration <= 120, "Fair",
  1=1, "Poor"
)
| sort -avg_duration
```

4. Data Quality Scoring

```
# Create a comprehensive data quality score
index=wineventlog source="WinEventLog:Security" EventCode=4624 earliest=-24h
| eval has_required_fields=if(
  isnotnull(Account_Name) AND Account_Name!=" AND
  isnotnull(Source_Network_Address) AND Source_Network_Address!=" AND Source_Network_Address!="-
  isnotnull(Computer_Name) AND Computer_Name!="",
  1, 0
)
| eval has_valid_ip=if(match(Source_Network_Address, "^d+\\.d+\\.d+\\.d+$"), 1, 0)
| eval has_valid_user=if(match(Account_Name, "^[A-Za-z0-9@\\._-]+$") AND len(Account_Name) > 0, 1, 0)
| eval has_valid_computer=if(match(Computer_Name, "^[A-Za-z0-9._-]+$") AND len(Computer_Name) > 0, 1, 0)
| stats count,
  avg(has_required_fields) as pct_complete,
  avg(has_valid_ip) as pct_valid_ip,
  avg(has_valid_user) as pct_valid_user,
  avg(has_valid_computer) as pct_valid_computer
| eval overall_quality_score=round((pct_complete + pct_valid_ip + pct_valid_user + pct_valid_computer)/4*100, 2)
```

Integration Testing

1. UBA Connectivity Test

```
# Test UBA connectivity to Splunk
curl -k -u uba_service:password \
  "https://splunk-server:8089/services/data/indexes" \
  -H "Accept: application/json"
```

2. Sample Data Retrieval Test

```
# Test sample data retrieval
curl -k -u uba_service:password \
  "https://splunk-server:8089/services/search/jobs" \
  -d "search=search index=wineventlog sourcetype=\"WinEventLog:Security\" EventCode=4624 | head 10 | eval dataFormat=\"AD\" | fields _time, Account_Name, Source_Network_Address, Computer_Name, EventCode, dataFormat" \
  -d "earliest_time=-1h" \
  -d "latest_time=now"
```

3. Field Extraction Validation

```
# Validate that UBA-required fields are properly extracted
index=wineventlog source="WinEventLog:Security" EventCode=4624 earliest=-15m
| eval dataFormat="AD"
| eval user=Account_Name
| eval src_ip=Source_Network_Address
| eval dest=Computer_Name
| eval signature=EventCode
| eval action="success"
| eval dataFormat="AD"
| where isnotnull(user) AND isnotnull(src_ip) AND isnotnull(dest)
| table _time, user, src_ip, dest, signature, action, dataFormat
| head 50
```

References

Primary Sources:

Splunk Inc. “Splunk User Behavior Analytics Admin Guide, Version 5.4.2.” Splunk Documentation, 2024. <https://docs.splunk.com/Documentation/UBA/5.4.2/GetDataIn/Intro>.

Splunk Inc. “Common Information Model (CIM) Add-on Manual.” Splunk Documentation, 2024. <https://docs.splunk.com/Documentation/CIM/5.3.2/User/Overview>.

Splunk Inc. “Splunk Enterprise Security Implementation Guide.” Splunk Documentation, 2024. <https://docs.splunk.com/Documentation/ES/7.3.0/Install/DeploymentPlanning>.

Secondary Sources:

Splunk Inc. “Search Processing Language (SPL) Reference.” Splunk Documentation, 2024. <https://docs.splunk.com/Documentation/Splunk/9.1.2/SearchReference/WhatsInThisManual>.

Splunk Inc. “Data Onboarding Best Practices.” Splunk Documentation, 2024. <https://docs.splunk.com/Documentation/Splunk/9.1.2/Data/Aboutgettingdatain>.

Splunk Inc. “Performance Monitoring and Tuning.” Splunk Documentation, 2024. <https://docs.splunk.com/Documentation/Splunk/9.1.2/Capacity/IntroductiontocapacityplanningforSplunk>.

Table of Sources

Source	Type	Description	URL
Splunk UBA Admin Guide 5.4.2	Official Documentation	Primary reference for UBA data source configuration	https://docs.splunk.com/Documentation/UBA/5.4.2/GetDataIn/Intro
Splunk CIM Add-on Manual	Official Documentation	Common Information Model compliance requirements	https://docs.splunk.com/Documentation/CIM/5.3.2/User/Overview
Splunk ES Implementation Guide	Official Documentation	Enterprise Security data source preparation	https://docs.splunk.com/Documentation/ES/7.3.0/Install/DeploymentPlanning
SPL Reference Manual	Official Documentation	Search Processing Language syntax and commands	https://docs.splunk.com/Documentation/Splunk/9.1.2/SearchReference/WhatsInThisManual
Data Onboarding Guide	Official Documentation	Best practices for data ingestion and preparation	https://docs.splunk.com/Documentation/Splunk/9.1.2/Data/Aboutgettingdatain
Performance Tuning Guide	Official Documentation	Capacity planning and performance optimization	https://docs.splunk.com/Documentation/Splunk/9.1.2/Capacity/IntroductiontocapacityplanningforSplunk

Document Version: 1.1
Created: June 18, 2025
Author: UBA Certificate Tools Suite
Last Updated: June 18, 2025